

Python Learning Guide: Making Decisions!

Mastering if, elif, and else statements

1. What Are Decisions in Code?

Every single day, you make dozens of decisions without even thinking about it! For example:

- **If** it rains today, you decide to take an umbrella.
- **If** it's sunny outside, you decide to wear your favorite sunglasses instead.
- **Otherwise** (if it's just normal weather), you simply go outside as you are!

Computers work exactly the same way. We use conditional statements to give our programs a brain so they can think, check conditions, and choose what action to take based on the facts.

2. The True and False Questions (Conditions)

Before a computer can make a decision, it must evaluate a **Condition**. A condition is simply a question that has only two possible answers: **True** or **False**.

Let's look at some real examples:

- Is `5 > 3`? → **True**
- Is `2 > 7`? → **False**
- Is `10 == 10`? → **True** (In Python, we use two equal signs `==` to ask if things are perfectly equal!)

Comparison Operators Cheat Sheet

To ask these questions, Python uses special symbols called comparison operators:

Operator	What It Means	Example
<code>==</code>	Equal to	<code>10 == 10</code> (True)
<code>!=</code>	Not equal to	<code>5 != 3</code> (True)
<code>></code>	Greater than	<code>12 > 20</code> (False)
<code><</code>	Less than	<code>4 < 9</code> (True)

<code>>=</code>	Greater than or equal to	<code>6 >= 6</code> (True)
<code><=</code>	Less than or equal to	<code>3 <= 2</code> (False)

3. The Three Heroes: if, elif, and else

Hero #1: The `if` Statement

The `if` keyword checks an individual condition. If that condition turns out to be **True**, the block of code hiding inside it runs perfectly!

```
age = 10
if age >= 6:
    print("You can go to school!")
```

Hero #2: The `else` Statement

What if your condition turns out to be **False**? The `else` statement acts as your backup plan. It triggers automatically when your `if` condition fails.

```
temperature = 15
if temperature > 30:
    print("It is hot!")
else:
    print("It is cool!")
```

Hero #3: The `elif` Statement

What if you have *more than just two alternatives*? Short for "else if", `elif` allows you to chain multiple options together perfectly. Python evaluates them one by one from top to bottom!

```
marks = 75
if marks >= 90:
    print("Excellent!")
elif marks >= 60:
    print("Good job!")
else:
    print("Keep trying!")
```

Important Rule: Python runs conditions from top to bottom. As soon as it finds ONE condition that is True, it runs that specific block and completely skips the rest! Only one block ever runs.

4. Critical Formatting Rules to Remember

- **Don't Forget the Colon (:):** You must always put a colon at the end of every `if`, `elif`, and `else` line.
- **Indentation Matters!** The code that is supposed to run inside your decision blocks **MUST** be pushed inward by **4 spaces**. If you forget to indent, Python will give you an error!

5. Combining Questions & Nesting

You can combine conditions using `and` (both must be true) or `or` (at least one must be true):

```
age = 10
has_ticket = True

if age >= 6 and has_ticket:
    print("Welcome to the show!")
```

You can also place an `if` statement directly inside another `if` statement! This is called **nesting**:

```
has_money = True
price = 20

if has_money:
    if price <= 50:
        print("You can buy it!")
    else:
        print("Too expensive!")
```



Practice Challenge!

Now it's your turn to be a brilliant developer! Write a Python program that creates a variable called `number`. Use your new skills to check and print whether that number is **positive**, **negative**, or **zero**.